

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas
Organización de Lenguajes y Compiladores 2
Vacaciones del primer semestre 2024
Catedrático: MSc. Luis Fernando Espino Barrios
Tutor Académico: José Andres Rodas Arrecis



Proyecto Fase 1

ARM64 Simulator

1. Competencias.....	1
1.1. Competencia General.....	1
1.2. Competencias Específicas.....	1
2. Descripción.....	1
2.1. Componentes de la aplicación.....	2
2.1.1. Editor.....	2
2.1.2. Funcionalidades.....	2
2.1.3. Herramientas.....	2
2.1.4. Reportes.....	2
2.1.5. Consola.....	3
2.2. Flujo del proyecto.....	3
2.3. Arquitectura.....	5
2.3.1. Aplicación Web Estática.....	5
3. Generalidades del lenguaje.....	5
3.1 Código Assembler ARM64.....	6
3.2 Comentarios.....	6
3.3 Registros.....	6
3.3.1 Registros de propósito general:.....	6
3.3.2 Registros de punto flotante:.....	7
3.3.3 Registro de Enlace de Instrucción:.....	7
3.3.4 Registros de Pila:.....	8
3.4 Etiquetas.....	9
3.5 Saltos.....	10
3.5 Operadores.....	11
3.5.1 Operadores Aritméticos:.....	11
3.5.2 Operadores Lógicos:.....	12
3.5.3 Operadores Relacionales:.....	14
3.5.4 Operadores de Desplazamiento:.....	15
3.6 Resumen de instrucciones.....	16
3.7 Ejemplo de Entrada.....	18
4. Reportes Generales.....	20

4.1. Reporte de errores.....	20
4.2. Reporte de CST.....	20
5. Entregables.....	21
6. Restricciones.....	21
7. Consideraciones.....	21
8. Entrega del proyecto.....	22

1.Competencias

1.1. Competencia General

Desarrollar un simulador para el lenguaje de programación **ensamblador ARM64** que abarque la fase de análisis, comprendiendo la estructura del código fuente utilizando herramientas adecuadas para la generación de analizadores léxicos y sintácticos.

1.2. Competencias Específicas

- Utilizar herramientas léxicas y sintácticas, como Peggy en Javascript, para implementar el reconocimiento y análisis del código fuente de ARM64, permitiendo al estudiante comprender la estructura del lenguaje y su sintaxis.
- Implementar correctamente las gramáticas adecuadas para el simulador y el correcto reconocimiento del lenguaje.
- Familiarizar al estudiante con las herramientas y estructuras de datos disponibles para la creación de un simulador, proporcionando una comprensión profunda de los procesos involucrados en la traducción y ejecución de programas escritos en lenguaje ensamblador arquitectura ARM64.

2. Descripción

La presente sección proporciona una visión general del proyecto que se solicita a los estudiantes, centrándose en el desarrollo de un simulador. Aunque la definición del lenguaje específico se abordará en secciones posteriores, se espera que los estudiantes trabajen con el lenguaje ensamblador para la arquitectura ARM64. Para lograr este propósito, se emplea la herramienta **Peggy** para el análisis sintáctico y léxico.

Este simulador tiene una estructura particular, ya que no se enfocará directamente en la interpretación del lenguaje como tal, en su lugar únicamente se validará la correcta estructura del lenguaje tomando en cuenta su sintaxis.

El proyecto requerirá que los estudiantes adquieran un entendimiento profundo de los principios fundamentales de la compilación del lenguaje ensamblador, así como de los aspectos específicos de la validación del código ensamblador para la arquitectura ARM64.

Este enfoque permite a los estudiantes explorar de manera práctica cómo se realiza la traducción de un lenguaje de alto nivel a instrucciones de bajo nivel en una arquitectura de conjunto de instrucciones reducido.

2.1. Componentes de la aplicación

El objetivo es desarrollar un entorno de desarrollo integral para la creación de aplicaciones en el lenguaje ensamblador ARM64. Este entorno, también conocido como Simulador, estará diseñado para analizar el código fuente escrito en ARM64 y proporcionar herramientas esenciales para facilitar el desarrollo de software.

La aplicación contará con los siguientes componentes:

2.1.1. Editor

El editor de código constituirá el núcleo del simulador, permitiendo al usuario ingresar y modificar el código fuente. Entre sus características destacadas se incluyen:

2.1.2. Funcionalidades

- Creación de archivos en blanco.
- Apertura de archivos con extensión .s.
- Guardado de archivos con extensión .s.
- Soporte para múltiples archivos abiertos simultáneamente.
- Visualización de la línea actual en el código.

2.1.3. Herramientas

Este conjunto de herramientas proporcionará funcionalidades adicionales para facilitar el desarrollo y análisis del código ensamblador ARM64.

- Validación del código fuente.
 - Botón que llama al simulador ARM64 para llevar a cabo los análisis léxico y sintáctico del código. Además deberá generar un mensaje que indique el resultado de la validación del código.

2.1.4. Reportes

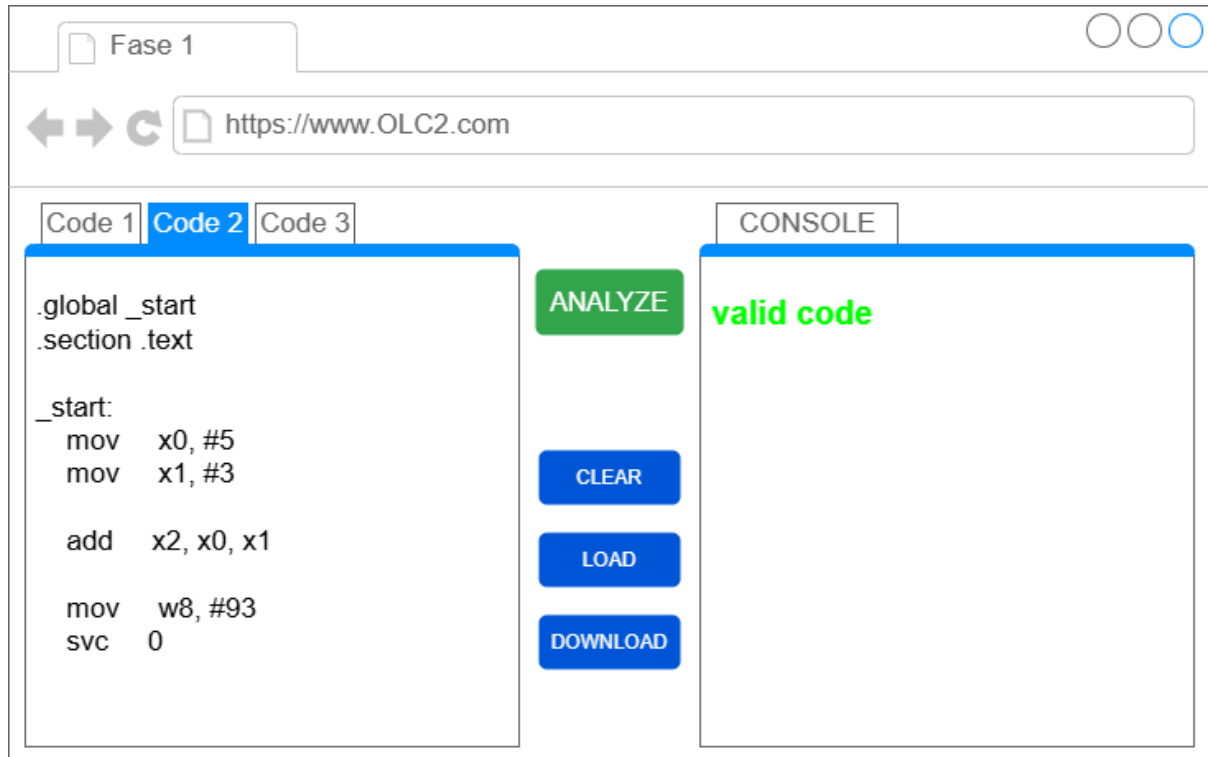
Reporte de Errores: Se mostrarán todos los errores encontrados al realizar el análisis léxico y sintáctico

Reporte de CST: Se mostrará una gráfica en donde se detalle el árbol cst generado al analizar sintácticamente el código fuente.

2.1.5. Consola

La consola proporcionará un espacio dedicado para visualizar notificaciones, errores, advertencias e impresiones generadas durante el proceso de análisis del código de entrada. Para esta fase del proyecto únicamente se debe mostrar si el código es válido o inválido tras el análisis sintáctico.

Interfaz propuesta: El simulador deberá contar como mínimo con los siguientes elementos mostrados en la siguiente interfaz.



(Interfaz propuesta)

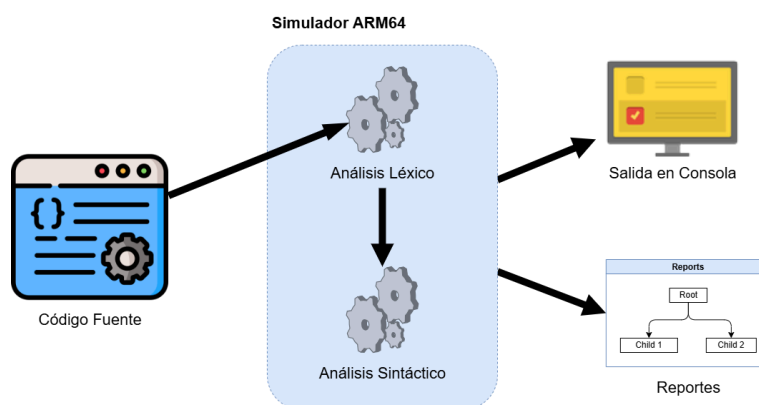
2.2. Flujo del proyecto

La ejecución del proyecto sigue un flujo definido según los procesos que los usuarios realizan durante su interacción con el sistema. A continuación, se presenta un desglose paso a paso del flujo del proyecto:

1. Inicio del Programa:
 - El usuario inicia el programa desde la aplicación web.
2. Código Fuente:
 - El usuario tiene la opción de cargar un archivo existente que contiene el código ensamblador o crear uno nuevo desde cero.
 - Si elige cargar un archivo existente, se abre el documento en el editor de texto integrado.

- Si decide crear un nuevo archivo, se inicia un nuevo documento en blanco en el editor.
 - El código fuente puede contener todas las instrucciones disponibles para el lenguaje ensamblador ARM64 descritas más adelante.
3. Análisis y ejecución: En el caso del simulador contará con las siguientes etapas del compilador:
- Análisis Léxico: El simulador escanea el código fuente y lo divide en tokens (unidades léxicas como palabras clave, identificadores, operadores, etc.).
 - Análisis Sintáctico: Utiliza la gramática del lenguaje ensamblador ARM64 para verificar la estructura sintáctica del código y construir un árbol de análisis sintáctico (AST).
4. Validación de Resultados:
- Se verifica que el programa cuente con la estructura sintáctica correcta, por lo que se debe mostrar un mensaje de validación en la consola de salida, asegurando así la precisión y la funcionalidad del código generado.
5. Generación de Reportes:
- Se generan informes detallados sobre los errores encontrados durante el análisis, incluyendo errores léxicos y sintácticos.
 - Además, se produce un gráfico en donde se detalla la estructura del árbol CST el cual debe contener todas las producciones de la entrada en cuestión.
6. Fin del Programa:
- Una vez completadas todas las etapas, el usuario puede cerrar el programa.

Este flujo del proyecto proporciona una guía clara sobre cómo los usuarios interactúan con la aplicación y cómo se llevarán a cabo los procesos clave durante el desarrollo y análisis del código ensamblador ARM64.



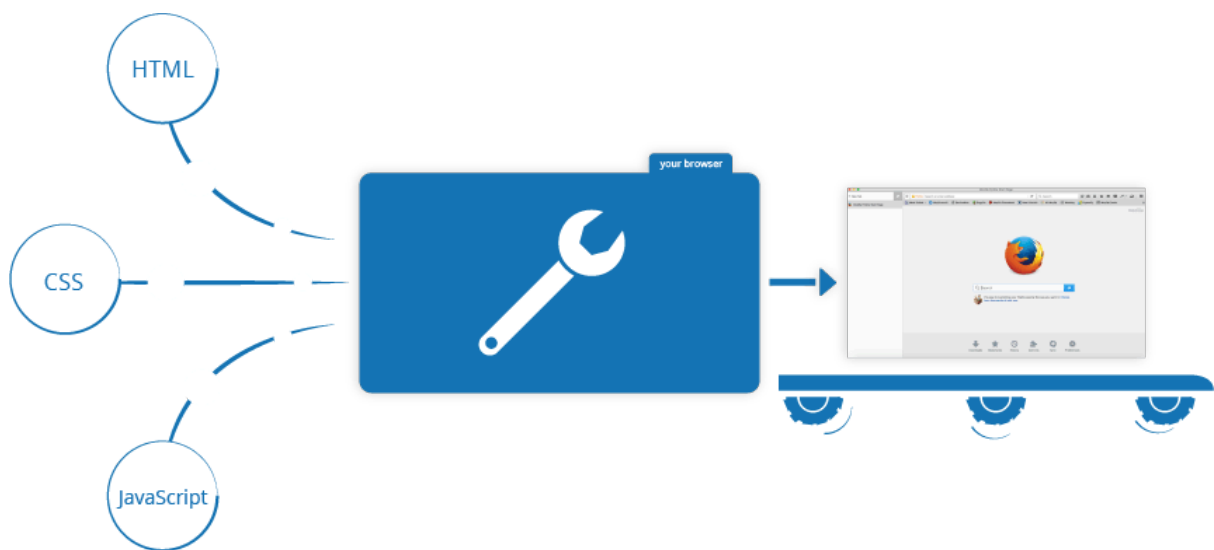
Flujo de ejecución del proyecto

2.3. Arquitectura

Para la implementación de la solución se requiere la siguiente arquitectura:

2.3.1. Aplicación Web Estática

Una aplicación web estática es una aplicación web cuya estructura, contenido y comportamiento no cambian dinámicamente en el lado del servidor. Estas aplicaciones se componen de archivos HTML, CSS y JavaScript que se entregan directamente al navegador del usuario tal como están almacenados en el servidor. Por lo tanto no estará permitido el uso de ningún framework de desarrollo como React, Angular, etc.



3. Generalidades del lenguaje

El lenguaje ensamblador que se evaluará a lo largo del proyecto se conforma por un subconjunto de instrucciones de este, con la diferencia de que el utilizado en esta fase tendrá una sintaxis más reducida pero sin perder las funcionalidades que caracterizan al lenguaje ensamblador ARM64 original.

Para verificar la salida del simulador se deberá implementar la herramienta Peggy de la mano con Javascript. Peggy es una biblioteca que permite definir gramáticas para lenguajes de programación o formatos de datos y generar parsers a partir de esas gramáticas. Utiliza la técnica de Parsing Expression Grammar (PEG), que es un enfoque formal para especificar gramáticas de lenguajes, similar a las expresiones regulares pero más potente y expresivo. Por lo tanto se solicita la creación de un analizador léxico (scanner) y un analizador sintáctico (parser) mediante dicha herramienta para realizar la validación del código de entrada en lenguaje ensamblador ARM64.

3.1 Código Assembler ARM64

El procesamiento de código assembler para la arquitectura ARM64 implica el análisis de las estructuras sintácticas y léxicas del lenguaje. Durante este proceso, se mapean las construcciones del lenguaje, como declaraciones de variables, expresiones aritméticas, control de flujo y saltos de etiquetas.

- Para acceder a la documentación de ARM64 utilizar el siguiente enlace: <https://developer.arm.com/documentation/ddi0553/latest/>
- Para acceder al listado de llamadas del sistema disponibles utilizar el siguiente enlace: https://chromium.googlesource.com/chromiumos/docs/+/_/master/constants/syscalls.md#arm64-64_bit

3.2 Comentarios

Un comentario es un componente léxico del lenguaje que no es tomado en cuenta para el análisis de la entrada. Se utilizará el formato del mismo ARM64 para comentarios:

- Los comentarios de una línea que serán delimitados al inicio con los símbolos “//” o “;” y al final con un carácter de finalización de línea.

```
// Este es un comentario en ARM64

MOV X0, #1 // Cargar el valor 1 en el registro X0

MOV X0, #1 ; Cargar el valor 1 en el registro X0
```

En este ejemplo, las líneas que comienzan con // y ; son comentarios y no tienen ningún impacto en el código ensamblador. Los comentarios son útiles para explicar el código y hacer anotaciones.

3.3 Registros

En ensamblador ARM64, los tipos de datos generalmente se manejan a nivel de registros y direcciones de memoria. No hay tipos de datos explícitos como en lenguajes de más alto nivel. Sin embargo, podemos considerar los registros como almacenando diferentes tipos de datos según su uso.

3.3.1 Registros de propósito general:

ARM64 cuenta con 31 registros de propósito general, denominados desde x0 hasta x30. Estos registros se utilizan para almacenar datos temporales, direcciones de memoria,

valores de retorno de funciones y otros propósitos generales. Los primeros 8 registros (x0 a x7) tienen funciones especiales en algunas instrucciones, como argumentos de función y valores de retorno.

```
# Cargar un entero en el registro x10
MOV x10, #42

# Cargar una dirección de memoria en el registro x11
MOV x11, mi_etiqueta
```

3.3.2 Registros de punto flotante:

ARM64 tiene 32 registros de punto flotante, denominados desde v0 hasta v31. Estos registros se utilizan para operaciones de punto flotante, como multiplicación, división y operaciones trigonométricas. Los registros de punto flotante se pueden acceder como registros individuales o en pares (doble precisión).

```
// Cargar los números de punto flotante en los registros
fmov d0, #10.5 // v0 = 10.5
fmov d1, #20.75 // v1 = 20.75
```

3.3.3 Registro de Enlace de Instrucción:

El registro de enlace de instrucción (LR) se utiliza comúnmente para almacenar la dirección de retorno cuando se llama a una subrutina. A continuación se muestra un ejemplo simple de cómo se puede utilizar LR.

```
.global _start
.section .text

_start:
    bl my_function // Llamada a la subrutina my_function

    // Aquí el control volverá después de que my_function haya terminado

    // Finalizar el programa y salir
    mov x8, #93 // syscall number para exit
    mov x0, #0 // código de salida
    svc #0 // realizar llamada al sistema para salir

my_function:
```



```

mov lr, x30 // Almacenar la dirección de retorno en LR

// Realizar algunas operaciones en la subrutina

// Retornar a la dirección de llamada
ret

```

3.3.4 Registros de Pila:

ARM64 tiene dos registros de puntero de pila: el Puntero de Pila Principal (MSP) y el Puntero de Pila de Proceso (PSP). A continuación se muestra un ejemplo simple de cómo se pueden usar estos registros para manipular la pila.

```

.global _start

.section .text

_start:
    // Configurar el puntero de pila de proceso (PSP) como el puntero de
    // pila actual
    mrs x0, PSP

    // Incrementar el puntero de pila para reservar espacio para datos
    // locales
    add x0, x0, #32 // Por ejemplo, reservar 32 bytes

    // Guardar el puntero de pila actualizado de vuelta en PSP
    msr PSP, x0

    // Realizar operaciones con la pila aquí

    // Restaurar el puntero de pila al valor original
    mrs x0, PSP
    sub x0, x0, #32 // Retroceder 32 bytes
    msr PSP, x0

    // Finalizar el programa y salir
    mov x8, #93 // syscall number para exit
    mov x0, #0 // código de salida
    svc #0 // realizar llamada al sistema para salir

```

3.4 Etiquetas

Las etiquetas serán identificadores únicos que indican una ubicación específica en el código fuente, permitiéndonos realizar saltos a lo largo en el código en la ejecución, estas se definirán durante el proceso de compilación, el formato de la etiqueta está definido de la siguiente manera:

El siguiente ejemplo ilustra un bucle que cuenta desde 10 hasta 1 y luego llama a una subrutina para imprimir el valor actual.

```
.global _start

.section .data
    format_str: .asciz "Current value: %d\n"

.section .text

_start:
    mov x0, #10          // Inicializar el contador a 10

loop:
    cmp x0, #0           // Comparar x0 con 0
    beq end_loop         // Si x0 es 0, saltar al final del bucle

    // Llamar a la subrutina print_value
    bl print_value

    sub x0, x0, #1       // Decrementar el contador
    b loop               // Saltar al inicio del bucle

end_loop:
    // Finalizar el programa y salir
    mov x8, #93          // syscall number para exit
    mov x0, #0           // código de salida
    svc #0               // realizar llamada al sistema para salir

print_value:
    // Aquí se implementa la lógica para imprimir el valor actual
    ret                 // Retorna al llamador
```

Explicación del código:

Inicio del Programa:

- **_start**: es la etiqueta que marca el punto de entrada del programa.
- **mov x0, #10** inicializa el contador en 10.

Bucle:

- **loop**: es la etiqueta que marca el inicio del bucle.
- **cmp x0, #0** compara el valor del contador con 0.
- **beq end_loop** salta a la etiqueta end_loop si el contador es 0.
- **bl print_value** llama a la subrutina print_value para manejar el valor actual del contador.
- **sub x0, x0, #1** decrementa el valor del contador.
- **b loop** salta de nuevo al inicio del bucle para repetir el proceso.

Fin del Bucle:

- **end_loop**: es la etiqueta que marca el final del bucle.
- **mov x8, #93** prepara la llamada al sistema para salir del programa.
- **mov x0, #0** establece el código de salida en 0.
- **svc #0** realiza la llamada al sistema para terminar el programa.

Subrutina print_value:

- **print_value**: es la etiqueta que marca el inicio de la subrutina.
- **ret** retorna al punto donde se llamó a la subrutina.

3.5 Saltos

Los saltos son instrucciones que permiten cambiar el flujo de ejecución del programa. En ARM64, los saltos condicionales e incondicionales se realizan mediante distintas instrucciones.

Instrucciones de salto condicional: Los saltos condicionales se utilizan para dirigir el flujo del programa en función del resultado de una comparación.

```
.global _start
.section

.text _start:

    mov x0, #10 // Cargar un número en x0

    cmp x0, #0  // Comparar el número con 0

    b.eq zero_label // Si x0 == 0, saltar a zero_label
    b.gt positive_label // Si x0 > 0, saltar a positive_label
    b.lt negative_label // Si x0 < 0, saltar a negative_label
```

Explicación del código:

Comparación y Saltos Condicionales:

- **cmp x0, #0**: Compara el valor del registro x0 con 0.
- **b.eq zero_label**: Salta a zero_label si x0 es igual a 0.
- **b.gt positive_label**: Salta a positive_label si x0 es mayor que 0.
- **b.lt negative_label**: Salta a negative_label si x0 es menor que 0.

Instrucciones de salto incondicional: Estas instrucciones siempre realizan un salto a la ubicación especificada, independientemente de alguna condición.

```
.global _start
.section .text

_start:
    mov x0, #5
    add x0, x0, #1    // x0 = x0 + 1

    // Salto incondicional a etiqueta skip_code
    b skip_code
```

3.5 Operadores

A continuación se detallan algunas de las operaciones aritméticas básicas que se pueden realizar en lenguaje ensamblador ARM64. Dependiendo de la extensión específica y de las características del conjunto de instrucciones, puede haber otras operaciones aritméticas disponibles, pero las principales son las siguientes:

3.5.1 Operadores Aritméticos:

Suma:

```
.global _start

.section .text

_start:
    mov x0, #10    // Cargar 10 en x0
    mov x1, #20    // Cargar 20 en x1
    add x2, x0, x1 // x2 = x0 + x1 (suma)
```

Resta:

```
.global _start
```

```
.section .text

_start:
    mov x0, #30      // Cargar 30 en x0
    mov x1, #15      // Cargar 15 en x1
    sub x2, x0, x1    // x2 = x0 - x1 (resta)
```

Multiplicación: En este ejemplo, `mul` multiplica el contenido de `x0` y `x1`, y almacena el resultado en `x2`.

```
.global _start

.section .text

_start:
    mov x0, #5        // Cargar 5 en x0
    mov x1, #6        // Cargar 6 en x1
    mul x2, x0, x1     // x2 = x0 * x1 (multiplicación)
```

División: La instrucción `udiv` realiza una división sin signo y `sdiv` realiza una división con signo.

```
.global _start

.section .text

_start:
    mov x0, #30      // Cargar 30 en x0
    mov x1, #5       // Cargar 5 en x1
    udiv x2, x0, x1   // x2 = x0 / x1 (división sin signo)
```

3.5.2 Operadores Lógicos:

AND:

```
.global _start
```

```
.section .text

_start:
    mov x0, #0b1100    // Cargar 0b1100 (12 en decimal) en x0
    mov x1, #0b1010    // Cargar 0b1010 (10 en decimal) en x1
    and x2, x0, x1     // x2 = x0 & x1 (AND)
```

OR:

```
.global _start

.section .text

_start:
    mov x0, #0b1100    // Cargar 0b1100 (12 en decimal) en x0
    mov x1, #0b1010    // Cargar 0b1010 (10 en decimal) en x1
    orr x2, x0, x1     // x2 = x0 | x1 (OR)
```

XOR:

```
.global _start

.section .text

_start:
    mov x0, #0b1100    // Cargar 0b1100 (12 en decimal) en x0
    mov x1, #0b1010    // Cargar 0b1010 (10 en decimal) en x1
    eor x2, x0, x1     // x2 = x0 ^ x1 (XOR)
```

NOT: La instrucción mvn (Move NOT) invierte todos los bits de un registro.

```
.global _start

.section .text

_start:
    mov x0, #0b1100    // Cargar 0b1100 (12 en decimal) en x0
    mvn x1, x0         // x1 = ~x0 (NOT)
```

3.5.3 Operadores Relacionales:

Igualdad:

```
.global _start

.section .text

_start:
    mov x0, #10        // Cargar 10 en x0
    mov x1, #10        // Cargar 10 en x1
    cmp x0, x1         // Comparar x0 con x1
    b.eq equal_label   // Si x0 == x1, saltar a equal_label

equal_label:
    // Código para cuando x0 es igual a x1
```

Desigualdad:

```
.global _start

.section .text

_start:
    mov x0, #10        // Cargar 10 en x0
    mov x1, #20        // Cargar 20 en x1
    cmp x0, x1         // Comparar x0 con x1
    b.ne not_equal_label // Si x0 != x1, saltar a not_equal_label

not_equal_label:
    // Código para cuando x0 es diferente de x1
```

Menor que:

```
.global _start

.section .text

_start:
    mov x0, #10        // Cargar 10 en x0
    mov x1, #20        // Cargar 20 en x1
    cmp x0, x1         // Comparar x0 con x1
```

```
    b.lt less_than_label // Si x0 < x1, saltar a less_than_label

less_than_label:
    // Código para cuando x0 es menor que x1
```

Mayor que:

```
.global _start

.section .text

_start:
    mov x0, #20      // Cargar 20 en x0
    mov x1, #10      // Cargar 10 en x1
    cmp x0, x1       // Comparar x0 con x1
    b.gt greater_than_label // Si x0 > x1, saltar a greater_than_label

greater_than_label:
    // Código para cuando x0 es mayor que x1
```

3.5.4 Operadores de Desplazamiento:

Desplazamiento a la izquierda (Logical Shift Left):

```
.global _start

.section .text

_start:
    mov x0, #1        // Cargar 1 en x0
    lsl x1, x0, #3     // x1 = x0 << 3 (desplazar a la izquierda 3 bits)
```

Desplazamiento a la derecha (Logical Shift Right):

```
.global _start

.section .text

_start:
```



```

mov x0, #8          // Cargar 8 en x0
lsr x1, x0, #2      // x1 = x0 >> 2 (desplazar a la derecha 2 bits)

```

3.6 Resumen de instrucciones

A continuación se detalla un resumen de las instrucciones a utilizar en el desarrollo del proyecto:

Descripción	Ejemplo
Load (cargar datos)	
LDR: Carga un valor de una dirección de memoria a un registro.	LDR x0, [x1] // Cargar el valor de la dirección contenida en x1 a x0
LDRB: Carga un byte de una dirección de memoria a un registro.	LDRB w0, [x1] // Cargar un byte de la dirección en x1 a w0
LDP: Carga dos valores de dos direcciones de memoria consecutivas a dos registros.	LDP x0, x1, [x2] // Cargar dos valores de la dirección en x2 a x0 y x1
Store (almacenar datos)	
STR: Almacena un valor de un registro a una dirección de memoria.	STR x0, [x1] // Almacenar el valor de x0 en la dirección contenida en x1
STRB: Almacena un byte de un registro a una dirección de memoria.	STRB w0, [x1] // Almacenar un byte de w0 en la dirección contenida en x1
STP: Almacena dos valores de dos registros en dos direcciones de memoria consecutivas.	STP x0, x1, [x2] // Almacenar los valores de x0 y x1 en la dirección contenida en x2
Instrucciones Aritméticas	
ADD: Suma dos valores y almacena el resultado en un registro.	ADD x0, x1, x2 // $x0 = x1 + x2$
SUB: Resta un valor de otro y almacena el resultado en un registro.	SUB x0, x1, x2 // $x0 = x1 - x2$
MUL: Multiplica dos valores y almacena el resultado en un registro.	MUL x0, x1, x2 // $x0 = x1 * x2$
UDIV: Divide dos valores sin signo y almacena el resultado en un registro.	UDIV x0, x1, x2 // $x0 = x1 / x2$ (sin signo)

SDIV: Divide dos valores con signo y almacena el resultado en un registro.	SDIV x0, x1, x2 // x0 = x1 / x2 (con signo)
Instrucciones Lógicas	
AND: Realiza una operación AND bit a bit entre dos valores y almacena el resultado.	AND x0, x1, x2 // x0 = x1 & x2
ORR: Realiza una operación OR bit a bit entre dos valores y almacena el resultado.	ORR x0, x1, x2 // x0 = x1 x2
EOR: Realiza una operación XOR bit a bit entre dos valores y almacena el resultado.	EOR x0, x1, x2 // x0 = x1 ^ x2
MVN: Realiza una operación NOT bit a bit sobre un valor y almacena el resultado.	MVN x0, x1 // x0 = ~x1
Instrucciones de Desplazamiento y Rotación	
LSL: Desplazamiento lógico a la izquierda.	LSL x0, x1, #3 // x0 = x1 << 3
LSR: Desplazamiento lógico a la derecha.	LSR x0, x1, #2 // x0 = x1 >> 2
ASR: Desplazamiento aritmético a la derecha.	ASR x0, x1, #2 // x0 = x1 >> 2 (con signo)
ROR: Rotación de bits a la derecha.	ROR x0, x1, #3 // Rotar x1 3 bits a la derecha y almacenar en x0
Instrucciones de Salto y Rama	
B: Salta incondicionalmente a una etiqueta.	B etiqueta // Saltar a 'etiqueta'
BL: Salta a una subrutina y guarda la dirección de retorno.	BL etiqueta // Llamar a subrutina en 'etiqueta'
RET: Retorna de una subrutina utilizando el registro de enlace (LR).	RET // Retornar de subrutina
BEQ: Salta si es igual (flags Z = 1).	BEQ etiqueta // Saltar a 'etiqueta' si Z = 1
BNE: Salta si no es igual (flags Z = 0).	BNE etiqueta // Saltar a 'etiqueta' si Z = 0
BGT: Salta si es mayor (flags Z = 0 y N = V).	BGT etiqueta // Saltar a 'etiqueta' si Z = 0 y N = V
BLT: Salta si es menor.	BLT etiqueta // Saltar a 'etiqueta' si x0 < x1

3.7 Ejemplo de Entrada

Entrada:

```
.global _start

.section .data
    valor: .word 10 // Definición de variable con el valor 10

.section .text

_start:
    // Cargar la dirección de la variable 'valor' en el registro x0
    LDR x0, =valor

    // Cargar el valor almacenado en 'valor' en el registro x1
    LDR x1, [x0]

    // Sumar 5 al valor almacenado en x1
    ADD x1, x1, #5

    // Comparar el valor resultante con 20
    CMP x1, #20

    // Saltar a etiqueta 'es_mayor' si x1 es mayor que 20
    BGT es_mayor

    // Saltar a etiqueta 'es_igual' si x1 es igual a 20
    BEQ es_igual

    // Saltar a etiqueta 'es_menor' si x1 es menor que 20
    BLT es_menor

es_mayor:
    // Código a ejecutar si x1 es mayor que 20
    // Por ejemplo, imprimir un mensaje
    mov x8, #64           // syscall number para sys_write
    mov x0, #1           // file descriptor: stdout
    LDR x1, =mayor_mensaje // dirección del mensaje
    mov x2, #10          // longitud del mensaje
    svc #0               // syscall para escribir en stdout
    B fin                // Saltar al final del programa

es_igual:
    // Código a ejecutar si x1 es igual a 20
    // Por ejemplo, imprimir un mensaje
    mov x8, #64           // syscall number para sys_write
```

```

    mov x0, #1                // file descriptor: stdout
    LDR x1, =igual_mensaje    // dirección del mensaje
    mov x2, #11               // longitud del mensaje
    svc #0                    // syscall para escribir en stdout
    B fin                      // Saltar al final del programa

es_menor:
    // Código a ejecutar si x1 es menor que 20
    // Por ejemplo, imprimir un mensaje
    mov x8, #64                // syscall number para sys_write
    mov x0, #1                 // file descriptor: stdout
    LDR x1, =menor_mensaje     // dirección del mensaje
    mov x2, #11                // longitud del mensaje
    svc #0                     // syscall para escribir en stdout

fin:
    // Finalizar el programa y salir
    mov x8, #93                // syscall number para exit
    mov x0, #0                 // código de salida
    svc #0                     // realizar llamada al sistema para salir

.section .rodata
    mayor_mensaje: .ascii "El resultado es mayor que 20\n"
    igual_mensaje: .ascii "El resultado es igual a 20\n"
    menor_mensaje: .ascii "El resultado es menor que 20\n"

```

Este código realiza lo siguiente:

- Define una variable valor con el valor inicial de 10.
- Carga la dirección de memoria de valor en x0 y luego carga el valor almacenado en valor en x1.
- Suma 5 al valor en x1.
- Compara el valor resultante con 20 y salta a etiquetas diferentes según el resultado de la comparación.
- Cada etiqueta imprime un mensaje diferente en la consola dependiendo del resultado de la comparación.
- El programa finaliza con una llamada al sistema para salir.

4. Reportes Generales

Como se indicaba al inicio, el simulador genera una serie de reportes sobre el proceso de análisis de los archivos de entrada. Los reportes son los siguientes:

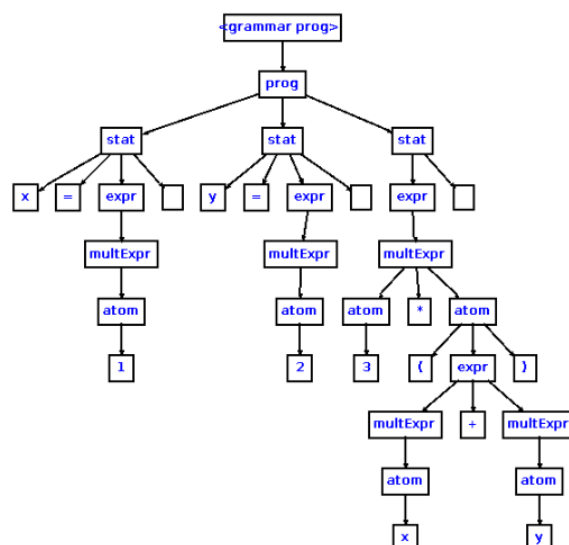
4.1. Reporte de errores

El simulador deberá ser capaz de detectar todos los errores que se encuentren durante el proceso de compilación. Todos los errores se deberán de recolectar y se mostrará un reporte de errores en el que, como mínimo, debe mostrarse el tipo de error, su ubicación y una breve descripción de por qué se produjo.

No	Descripción	Línea	Columna	Tipo
1	Se esperaba "ID" pero se obtuvo "(".	5	1	sintáctico
2	Se esperaba "ADD", "SUB" o "MUL" pero se obtuvo "}".	19	6	sintáctico
3	El símbolo "¬" no es aceptado en el lenguaje.	55	2	léxico

4.2. Reporte de CST

Este reporte mostrará el árbol CST generado al momento de ejecutar el análisis sintáctico del código de entrada. El árbol debe mostrarse de manera gráfica, con cada una de sus hojas y ramas debidamente identificadas.



5. Entregables

El estudiante deberá entregar únicamente el link de un repositorio en GitHub, el cual será privado y contendrá todos los archivos necesarios para la ejecución de la aplicación, así como el código fuente, la gramática y la documentación. El estudiante es responsable de verificar el contenido de los entregables, los cuales son:

- 5.1. Código fuente de la aplicación
- 5.2. Gramáticas utilizadas

Se deben conceder los permisos necesarios al auxiliar para acceder a dicho repositorio. El usuario es el siguiente:

- AndresRodas

6. Restricciones

- 6.1. El proyecto deberá realizarse como una aplicación web estática utilizando el lenguaje **Javascript**.
- 6.2. Es válido el uso de cualquier herramienta y/o librería en Javascript para el desarrollo de la interfaz gráfica a excepción de frameworks que afecten a la arquitectura del proyecto como Angular, Vue.js, React, etc.
- 6.3. Para el analizador léxico y sintáctico se debe implementar una gramática con la herramienta **Peggy** de Javascript.
- 6.4. Para la validación del código ensamblador se debe tener como base la arquitectura ARM64.
- 6.5. Las copias de proyectos tendrán de manera automática una nota de **0 puntos** y los involucrados serán reportados a la Escuela de Ciencias y Sistemas.
- 6.6. El desarrollo y la entrega del proyecto es en grupos con un máximo de 3 estudiantes.
- 6.7. La distribución de los grupos queda a discreción de los estudiantes.
- 6.8. El sistema operativo queda a elección del estudiante.

7. Consideraciones

- 7.1. Durante la calificación se realizarán preguntas sobre el código para verificar la autoría de este, de no responder correctamente la mayoría de las preguntas **el tutor hará una verificación exhaustiva en busca de copias**.
- 7.2. Se necesita que los estudiantes al momento de la calificación tengan el entorno de desarrollo y las herramientas necesarias para realizar compilaciones, ya que se pedirán pequeñas modificaciones en el código para verificar la autoría de este, en caso que los estudiantes no puedan realizar dichas modificaciones en un tiempo prudencial, los estudiantes tendrán 0 en la sección ponderada y **el tutor hará una verificación exhaustiva en busca de copias**.

- 7.3. Se tendrá un máximo de 30 minutos por estudiante para calificar el proyecto. La calificación será de manera virtual y se grabará para tener constancia o verificación posterior.
- 7.4. La hoja de calificación describe cada aspecto a calificar, por lo tanto, si la funcionalidad a calificar falla en la sección indicada se tendrá 0 puntos en esa funcionalidad y esa nota no podrá cambiar si dicha funcionalidad funciona en otra sección.
- 7.5. Si una función del programa ya ha sido calificada, esta no puede ser penalizada si en otra sección la función falla o es errónea.
- 7.6. Los archivos de entrada permitidos en la calificación son únicamente los archivos preparados por el tutor.
- 7.7. Los archivos de entrada podrán ser modificados solamente antes de iniciar la calificación eliminando funcionalidades que los estudiantes indiquen que no desarrollaron.
- 7.8. La sintaxis descrita en este documento son con fines descriptivos, el estudiante es libre de diseñar la gramática que crea apropiada para el reconocimiento del lenguaje ensamblador ARM64.

8. Entrega del proyecto

- 8.1. La entrega se realizará de manera virtual, se habilitará un apartado en la plataforma de UEDI para que el estudiante realice su entrega.
- 8.2. No se recibirán proyectos fuera de la fecha y hora estipulada.
- 8.3. La entrega de cada una de las fases del proyecto es en **grupos con máximo de 3 alumnos.**
- 8.4. Fecha límite de entrega del proyecto: **11 de junio de 2024 a las 7:59 hrs.**